Design and Analysis of Approximate Multiplier for Low-Power AI Applications

Under the esteemed guidance of

Mr. S. Balarama Murthy

Associate Professor

Presented by

Sesetti Gayathri

25NU1D7712

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

NADIMPALLI SATYANARAYANA RAJU INSTITUTE OF TECHNOLOGY (A)

INDEX

ABSTRACT

- AI and deep learning applications are growing rapidly and need hardware that is fast, small, and uses less power. Multiplication is the most commonly used and resource-heavy operation in AI systems.
- Approximate computing allows hardware to make small, bounded errors — since AI applications remain accurate despite tiny errors, approximate multipliers help reduce power, area, and hardware complexity.
- This project designs and analyzes a 4-bit × 4-bit approximate multiplier for low-power AI applications.
- An exact multiplier is first designed in Verilog HDL using the standard partial-product method, always producing the correct answer and serving as the reference for comparison.
- An approximate multiplier is then designed by removing some smaller partial products and keeping only the important larger ones, making the circuit simpler and more power-efficient.
- Both designs are tested using Icarus Verilog and GTKWave, synthesized on Xilinx Vivado to measure LUT usage, speed, and power, and compared using Python to calculate the error.
- The results show that the approximate multiplier offers a practical trade-off between hardware savings and accuracy, making it well suited for low-power AI hardware.

Keywords: *Approximate Computing, Approximate Multiplier, AI Applications, Partial Product Truncation, Verilog HDL, Low-Power VLSI.*

EXISTING WORK

- Base paper designs a low-power approximate 8×8 unsigned multiplier for deep neural network (DNN) applications, built around a custom 4:2 compressor using NOR/NAND-based logic for a short critical path.
- The compressor makes an error in only 1 out of 256 input combinations. Synthesized in Verilog HDL on Cadence Genus (UMC 90nm), it achieves up to 30.24% energy and 33.14% power savings over existing designs, with MRED of just 0.109%.
- Validated on real DNN tasks: MNIST digit recognition using LeNet-5 (98.24% exact vs 96.45% approximate) and FFDNet image denoising — confirming the approximation barely affects DNN performance.
- **Research Gap**: this accuracy needs a fully custom-redesigned compressor (Area 30.57 μm^2, Delay 237ps) verified on a commercial EDA flow (Cadence Genus, 90nm) — high design and verification cost, not achievable with open-source tools. An 8×8 design also needs exhaustive verification across 65,536 input combinations, which is heavy for open-source simulation flows — so this project targets a 4×4 multiplier, enabling full exhaustive verification (all 256 combinations) while validating the same truncation-based approximation technique, extensible to 8×8 and higher-order multipliers as future work.
- My Project achieves a similar trade-off with simple partial-product truncation: 37.5% LUT reduction, 17.0% delay reduction, and 45.7% power reduction over an exact multiplier — verified entirely on free, open-source tools (Icarus Verilog, GTKWave, Vivado WebPACK).

SOFTWARE TOOLS

- VS Code (Latest stable) — RTL and Python code editing.
- Icarus Verilog (10.3) — Verilog compilation and simulation.
- GTKWave (3.3.103) — Waveform inspection (VCD).
- Python 3 (3.8.10) with NumPy / Matplotlib (Latest) — Error analysis, numerical analysis, and plotting.
- Vivado WebPACK (2026.1) — Synthesis and timing/power reports. Installed via apt (Icarus Verilog, GTKWave, Python) and pip (NumPy, Matplotlib, pandas); verified with iverilog -V, gtkwave --version, and python3 --version.

EXACT MULTIPLIER — PARTIAL PRODUCT METHOD

- A 4-bit × 4-bit multiplication generates four partial products (pp0–pp3), each formed by shifting A according to a bit of B.
- All four partial products are summed to produce the 8-bit result — mathematically perfect for every input.
- Verified across all 256 possible 4-bit input combinations — zero error, 100% exhaustive coverage.
- Worked Example: 10 × 12 = 120 → pp2 = 00101000, pp3 = 01010000, Sum = 01111000 = 120.
- Serves as the golden reference used to measure the accuracy and hardware savings of the approximate design.

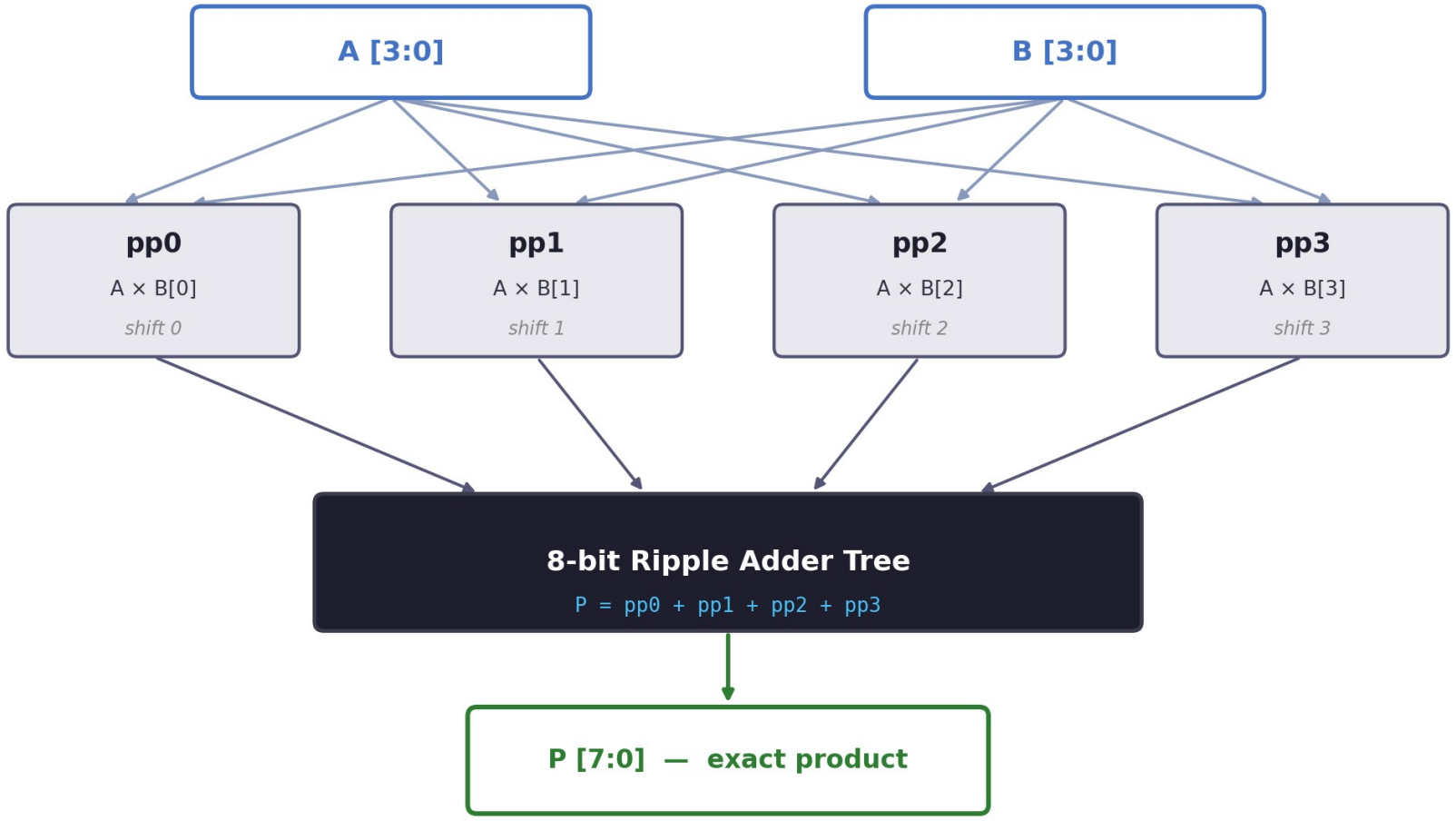
exact_multiplier.v

```
module exact_multiplier (
  input  [3:0] A, B,
  output [7:0] P
);

wire [7:0] pp0 = B[0] ?
  {4'b0, A} : 8'b0;
wire [7:0] pp1 = B[1] ?
  {3'b0, A, 1'b0} : 8'b0;
wire [7:0] pp2 = B[2] ?
  {2'b0, A, 2'b0} : 8'b0;
wire [7:0] pp3 = B[3] ?
  {1'b0, A, 3'b0} : 8'b0;

assign P = pp0+pp1+pp2+pp3;
endmodule
```

EXACT MULTIPLIER — BLOCK DIAGRAM



All four partial products fully summed — zero error across all 256 input combinations

APPROXIMATE MULTIPLIER — PARTIAL-PRODUCT TRUNCATION

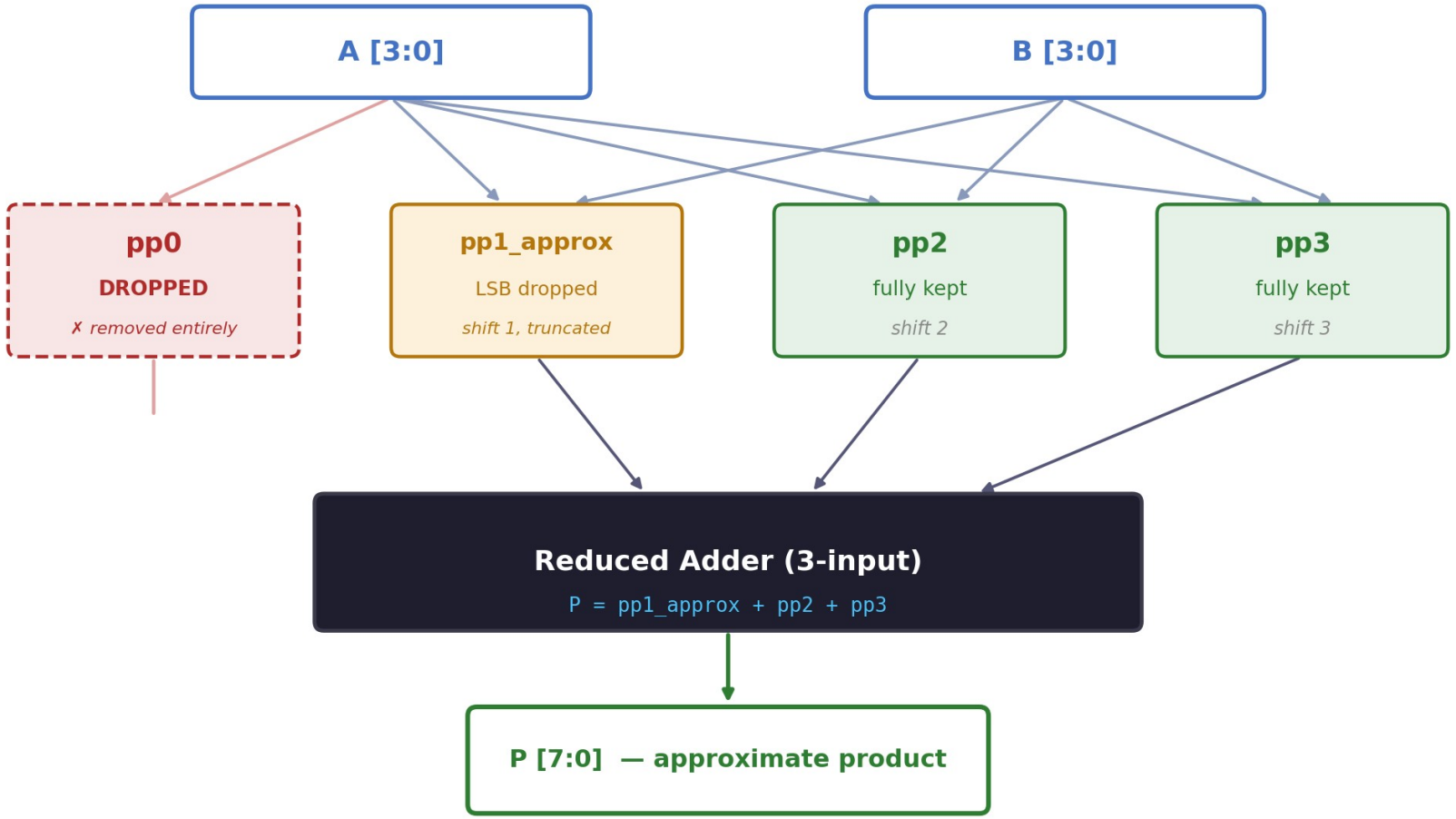
- The approximate multiplier drops pp0 entirely and truncates pp1, keeping pp2 and pp3 fully intact.
- pp3 & pp2: fully kept | pp1: truncated (LSB dropped) | pp0: dropped entirely.
- Removing one full-adder level shortens the critical path, reduces LUT usage, and lowers switching power.
- Both the exact and approximate multipliers share a common testbench so their outputs can be directly compared.

approx_multiplier.v

```
wire [7:0] pp3 = B[3] ?
  {1'b0,A,3'b0} : 8'b0;
wire [7:0] pp2 = B[2] ?
  {2'b0,A,2'b0} : 8'b0;
wire [7:0] pp1_approx = B[1] ?
  {3'b0,A[3:1],2'b0} : 8'b0;

// pp0 dropped
assign P = pp1_approx
         + pp2 + pp3;
```

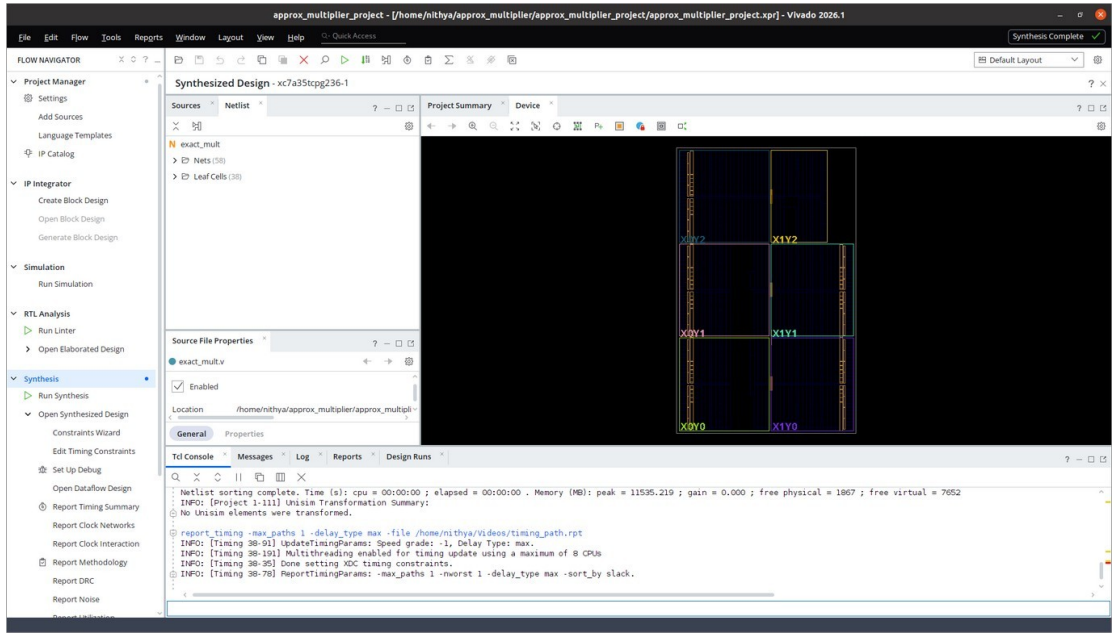
APPROXIMATE MULTIPLIER — BLOCK DIAGRAM



One full-adder level removed → shorter critical path, fewer LUTs, lower switching power

SYNTHESIS OUTPUTS

Vivado synthesized-design view — Exact Multiplier (Xilinx Artix-7, xc7a35tcpg236-1)

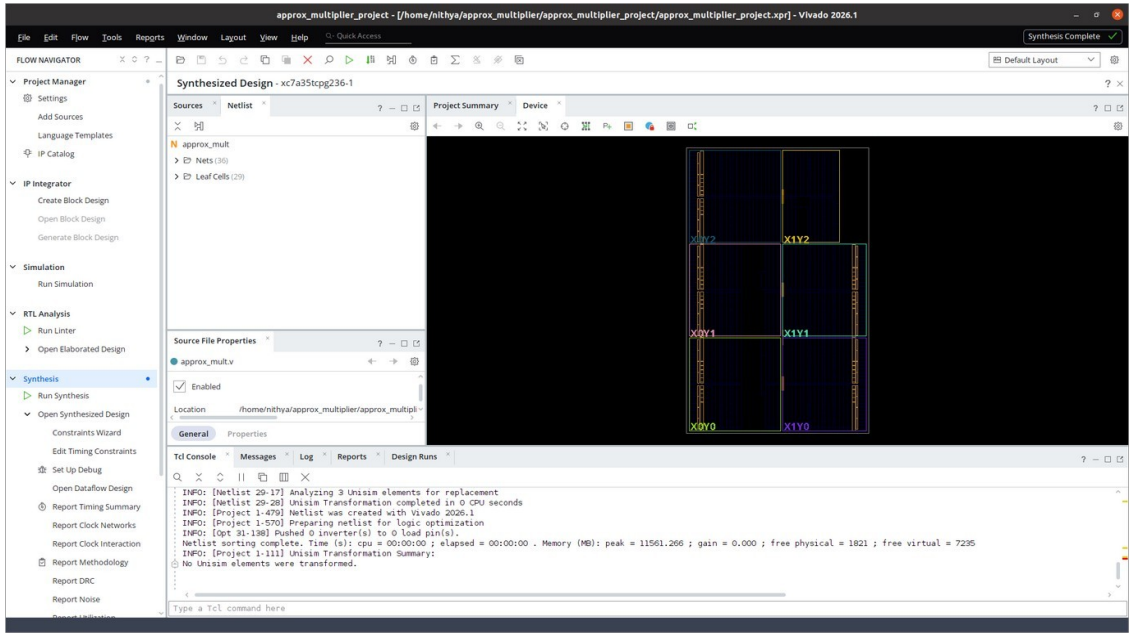


Exact Multiplier — Synthesized

- Leaf cells: 38 — synthesized cell count for the exact multiplier design.
- Nets: 58 — interconnect nets in the exact multiplier design.

SYNTHESIS OUTPUTS

Vivado synthesized-design view — Approximate Multiplier (Xilinx Artix-7, xc7a35tcpg236-1)

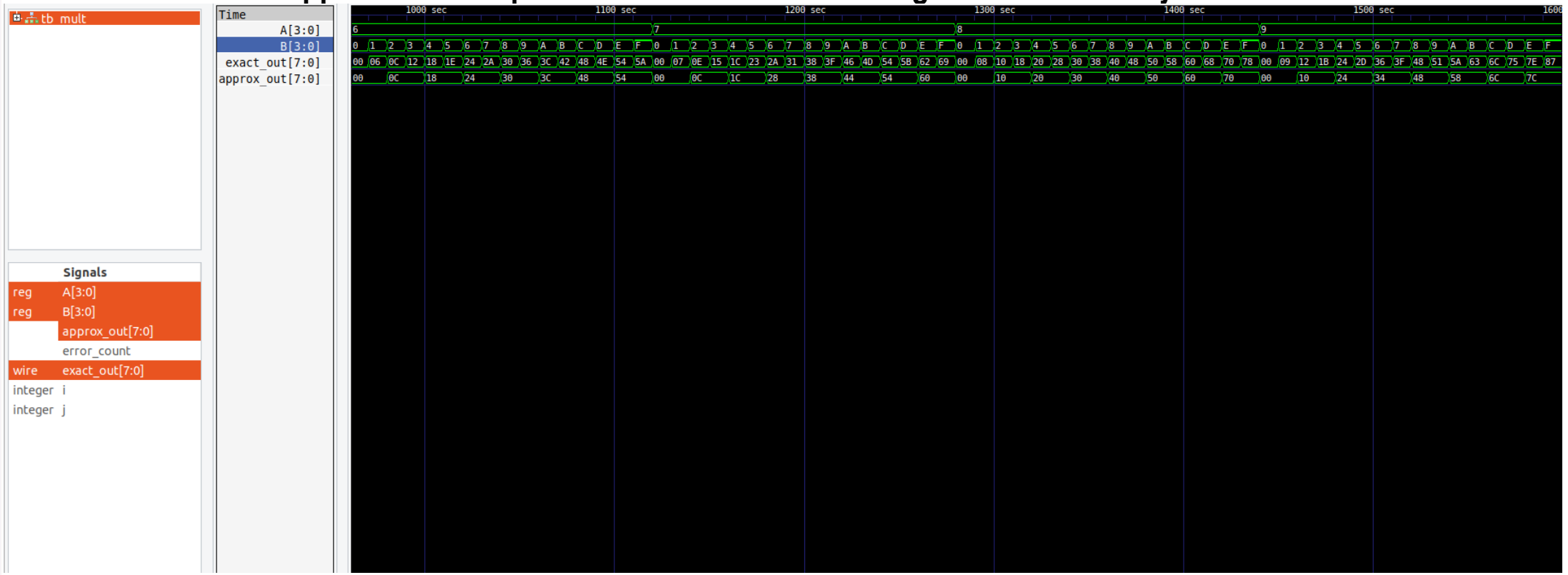


Approximate Multiplier — Synthesized

- Leaf cells: 29 — fewer synthesized cells confirm the reduced logic footprint (vs 38 for exact).
- Nets: 36 — simplified partial-product interconnect (vs 58 for exact).

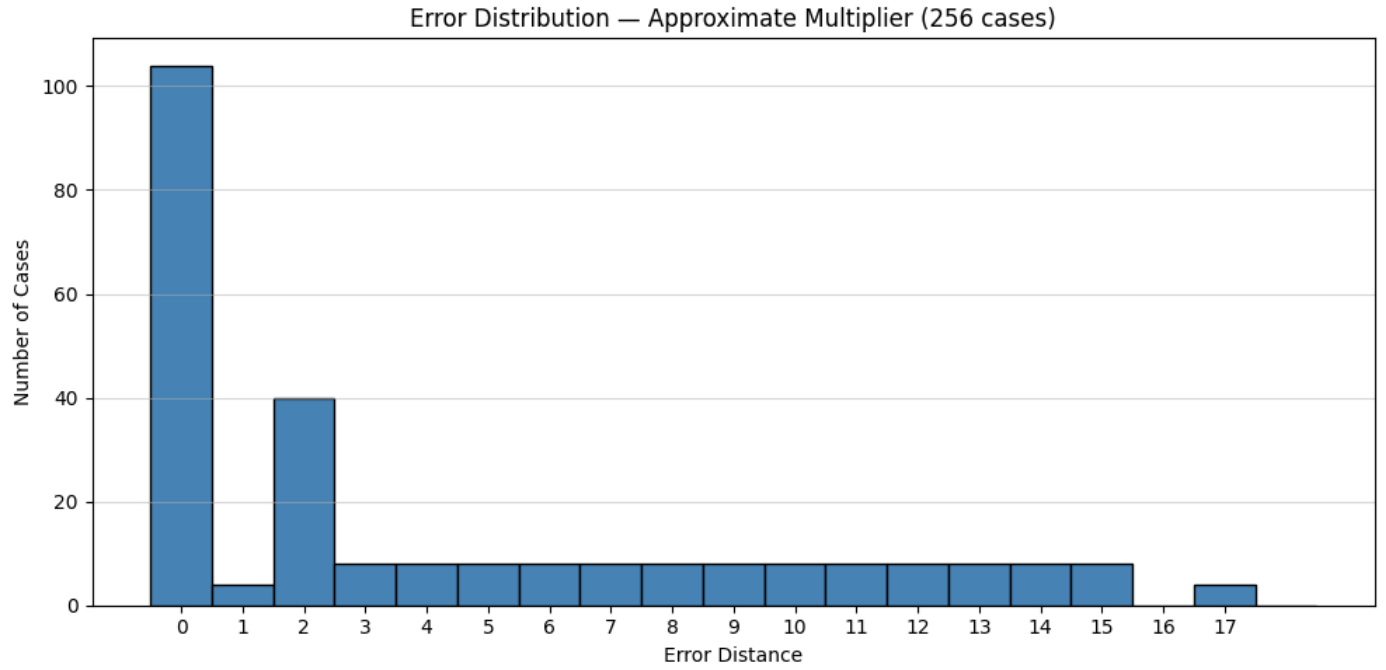
SIMULATION RESULTS — GTKWAVE VERIFICATION

Common testbench applies all 256 input combinations to both designs simultaneously



- Observation: Outputs match exactly for (3,5), (4,6), (6,6) — dropped bits don't contribute. Visible divergence appears for (1,8), (7,2), (15,15), (9,3), (12,11).

ERROR ANALYSIS — PYTHON METRICS



4.25
Mean Error Distance (MED)

59.38%
Error Rate (ER)

17
Max Error Distance

16.12%
Mean Relative Error (MRE)

- Errors increase with operand magnitude but remain bounded (Max Error Distance = 17) — a predictable pattern that AI workloads can tolerate.
- Max error (17) occurs at the largest operand pairs: (A=15, B=3), (15, 7), (15, 11), (15, 15).

SYNTHESIS RESULTS — HARDWARE COMPARISON

Synthesis results from Vivado 2026.1, target device xc7a35tcpg236-1 (Artix-7).

Metric	Exact	Approximate	Savings
LUTs	16	10	**37.5%**
Critical Path Delay (ns)	6.990	5.804	**17.0%**
Total On-Chip Power (W)	4.717	2.563	**45.7%**

- LUT count and Total On-Chip Power taken from report_utilization and report_power on the synthesized design.
- Critical Path Delay is the Data Path Delay from report_timing (max path), since both designs are purely combinational with no clock defined.
- The approximate multiplier cuts LUT usage by 37.5%, delay by 17.0%, and power by 45.7% compared with the exact design.

APPLICATIONS

- **Edge-AI Inference Accelerators —** smart cameras and surveillance systems performing on-device object detection (e.g., person or vehicle recognition) need fast, low-power hardware to avoid constant cloud offloading, extending battery life for always-on operation.
- **Low-Power IoT & Wearable Devices —** fitness bands and smartwatches running on-device health analytics, such as heart-rate anomaly or fall detection, operate on tiny batteries for days at a time, so every milliwatt saved in the multiplier directly extends usable battery life.
- **Real-Time Signal Processing —** voice-assistant wake-word detection (e.g., “Hey Siri”, “OK Google”) on hearables and smart speakers must continuously process audio streams within strict power and latency budgets, making approximate multipliers well suited for always-listening front ends.
- **Mobile Neural-Network Hardware —** smartphone NPUs handling everyday on-device AI tasks, such as face unlock, camera scene detection, and live translation, rely on millions of multiply operations per second; approximate multipliers cut power without noticeably affecting user-facing accuracy.

CONCLUSION & FUTURE WORK

Conclusion:

- Exact multiplier: zero error across all 256 test cases (golden reference).
- Approximate design: 37.5% fewer LUTs, 17.0% less delay, 45.7% less power.
- Error stays bounded (Max Error Distance = 17) and data-dependent.
- AI matrix-multiply demo confirms inference quality is still acceptable.
- **A small, controlled error buys big gains in speed, size, and power.**

Future Work:

- Scale the design to 8×8 and 16×16 multipliers.
- Explore signed multiplication and other approximate techniques.
- Validate the design on real FPGA hardware.

REFERENCES

[1] Sathiya, Prabhavathy, Balasupramani, Amutha, Kavitha, Saravanakumar, "Low Power Approximate Multiplier Architecture for Deep Neural Networks," 2025 International Conference on Visual Analytics and Data Visualization (ICVADV), pp. 541–548, 2025. DOI: 10.1109/ICVADV63329.2025.10961086. [Base Paper]

[2] V. Gupta, D. Mohapatra, S. P. Park, A. Raghunathan, K. Roy, "IMPACT: IMPrecise adders for low-power approximate computing," IEEE/ACM Int. Symp. on Low Power Electronics and Design (ISLPED), pp. 409–414, 2011. DOI: 10.1109/ISLPED.2011.5993675.

[3] Z. Aizaz, K. Khare, A. Tirmizi, "Efficient Approximate Multipliers for Neural Network Applications," in Computational Intelligence in Data Mining, Springer, 2022. DOI: 10.1007/978-981-16-9447-9_44.

[4] S. S. Sarwar, S. Venkataramani, A. Ankit, A. Raghunathan, K. Roy, "Energy-efficient neural computing with approximate multipliers," ACM Journal on Emerging Technologies in Computing Systems, vol. 14, no. 2, 2018.

[5] H. Waris, C. Wang, W. Liu, J. Han, F. Lombardi, "Hybrid partial product-based high-performance approximate recursive multipliers," IEEE Transactions on Emerging Topics in Computing, vol. 10, no. 1, pp. 507–513, 2022. DOI: 10.1109/TETC.2020.3013977.

[6] Q. Xu, T. Mytkowicz, N. S. Kim, "Approximate computing: A survey," IEEE Design & Test, vol. 33, no. 1, pp. 8–22, 2016. DOI: 10.1109/MDAT.2015.2505723.

[7] A. G. M. Strollo, E. Napoli, D. De Caro, N. Petra, G. D. Meo, "Comparison and extension of approximate 4–2 compressors for low-power approximate multipliers," IEEE Transactions on Circuits and Systems I, vol. 67, no. 9, pp. 3021–3034, 2020. DOI: 10.1109/TCSI.2020.2988353.

THANK YOU